Learning Objectives

- Understand how function arguments and return values can be passed using the stack
- Distinguish the responsibilities of caller and callee
- Understand the stack layout during function calls
- Extend the code generator to generate function definitions and function calls
- Generate code for passing arguments and returning values
- Integrate the code generator from Session 10 into our compiler project

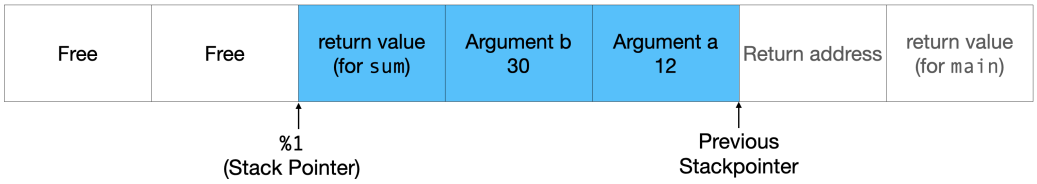
Revisiting Our Calling Convention from Worksheet 14

In Worksheet 14, we developed a calling convention for passing values to functions and returning results. Let's take another look at the stack layout for a call to `sum(12, 30)`.

First, a small distinction in terminology: **arguments** are the values supplied by the caller, whereas **parameters** are the variables through which the callee accesses these values. In our example, `12` and `30` are arguments, while `a` and `b` are parameters of sum.

Before the function call (caller's perspective):

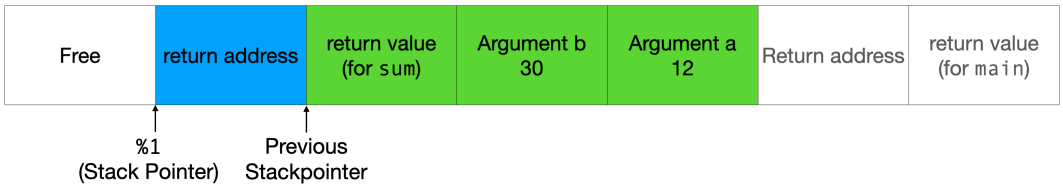
In Worksheet 14, we developed a calling convention for passing values to functions and returning



The caller has reserved space for the return value and the arguments. Relative to the current stack pointer `%1`, their offsets are:

- Return value: `0(%1)`
- Argument b: `8(%1)`
- Argument a: `16(%1)`

After the prologue (callee's perspective):



The callee has pushed the return address onto the stack. Consequently, the stack pointer has changed, and so have the offsets:

- Return value: `8(%1)`
- Parameter b: `16(%1)`
- Parameter a: `24(%1)`

Notice that the values are still stored at exactly the same memory addresses. Only the stack pointer has moved.

Now imagine that the callee could still access the **caller's stack pointer**, i.e., its value immediately before the call. The offsets for accessing parameters inside the function would then be identical to those used by the caller for passing arguments.

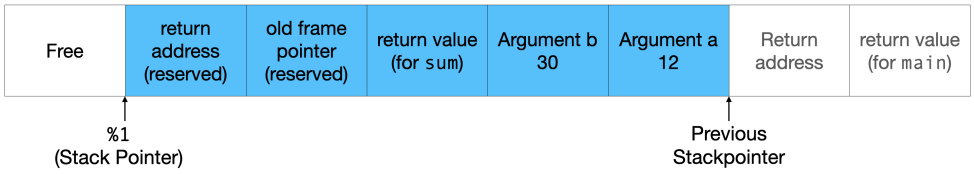
This would greatly simplify our compiler: we could store these offsets in the symbol table and reuse them both when passing arguments and when accessing parameters.
That is precisely the idea behind introducing a frame pointer.

Our New Stack Layout

Let's revisit our call to `sum(12, 30)`, this time using the new calling convention.

Before the function call (caller's perspective):

The caller now reserves space for two additional entries: the return address and the old frame pointer. The stack layout looks as follows:

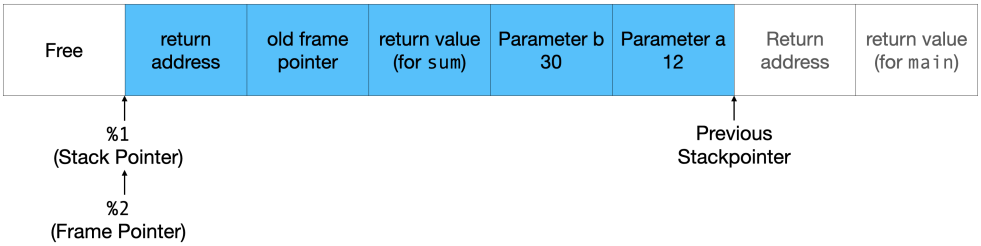


Relative to the stack pointer `%1`, the offsets are:

- Return address: `0(%1)`
- Old frame pointer: `8(%1)`
- Return value: `16(%1)`
- Argument `b`: `24(%1)`
- Argument `a`: `32(%1)`

After the prologue (callee's perspective)

The callee stores the return address and the old frame pointer in the reserved locations. It then copies `%1` into `%2`, establishing the new frame pointer.



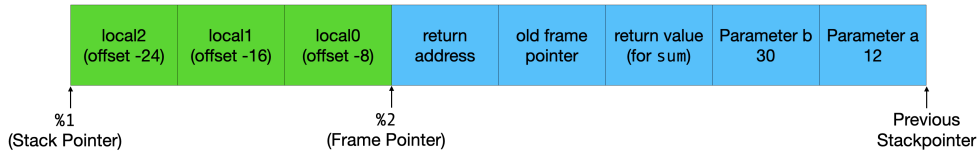
The stack pointer still points to the same location, and the frame pointer now points there as well. Consequently, the offsets remain unchanged:

- Return address: `0(%2)`
- Old frame pointer: `8(%2)`
- Return value: `16(%2)`
- Parameter `b`: `24(%2)`
- Parameter `a`: `32(%2)`

The important observation is that the caller uses offsets relative to the stack pointer, while the callee uses the same offsets relative to the frame pointer.
This will become particularly useful when we introduce local variables: the stack pointer may change during the execution of a function, but the frame pointer remains fixed.

Looking Ahead: Local Variables

Why do we need two pointers if `%1` and `%2` initially point to the same location? The stack pointer %1 may change, for example when the function calls another function. The frame pointer `%2`, however, remains fixed, allowing parameters to be accessed using constant offsets—regardless of changes to the stack pointer.
Later in this worksheet, we will also reserve stack space for local variables. This gives us the following stack layout:



Notice the simple pattern:

- **Parameters** have fixed, positive offsets relative to `%2`.
- **Local variables** have fixed, negative offsets relative to `%2`.

This is exactly what we need for our compiler: the symbol table can store a fixed offset for each parameter or local variable, and the code generator can use it directly whenever that variable is accessed.

Step 1 — Putting It All Together (Again!)

In Worksheet 14, we implemented the following program in ULM assembly:

```
fn sum(a: u64, b: u64): u64
{
    return a + b;
}

fn main(): u64
{
    return sum(12, 30);
}
```

Let's revisit this example using our new calling convention. The program should still terminate with exit code 42, but this time we will use %2 as the frame pointer.

Tasks

1. **Calling main.** Create a new assembly file ex1.s. Start with the following startup code:

```
.text
    loadz   0,      %1

    subq    24,     %1,     %1
    call    main,   %3
    movq    16(%1), %4
    addq    24,     %1,     %1
    halt    %4
```

   Notice that we now reserve 24 bytes: 8 bytes each for the return address, the old frame pointer, and the return value.

2. **Defining sum.** Next, add the following implementation of `sum`:

```
sum:
    movq    %3,     (%1)        # store return address
    movq    %2,     8(%1)       # save previous frame pointer
    movq    %1,     %2          # set new frame pointer

    // begin of function body
    movq    32(%2), %4
    movq    24(%2), %5
    addq    %4,     %5,     %4
    movq    %4,     16(%2)      # store %4 as return value
    // end of function body

    movq    %2,     %1          # restore stack pointer at function entry
    movq    8(%1),  %2          # restore previous frame pointer
    movq    (%1),   %3          # restore return address
    ret     %3
```

   The prologue saves the return address and the old frame pointer, then establishes `%2` as the new frame pointer. Unlike in Worksheet 14, the stack pointer does not need to be adjusted: the caller has already reserved the necessary space. The function body accesses both parameters using fixed offsets relative to `%2` and stores the result at `16(%2)`.
   The epilogue first restores `%1` to its value at function entry. It then restores the previous frame pointer and the return address. This works even if the stack pointer has changed during the execution of the function.

3. **Defining main.** Now add the following implementation of `main`:

```
main:
    movq    %3,     (%1)        # store return address
    movq    %2,     8(%1)       # save previous frame pointer
    movq    %1,     %2          # set new frame pointer

    /* function body here */

    movq    %2,     %1          # restore stack pointer at function entry
    movq    8(%1),  %2          # restore previous frame pointer
    movq    (%1),   %3          # restore return address
    ret     %3
```

   Notice that `main` uses exactly the same prologue and epilogue as `sum`. These patterns do not depend on the number of parameters.

4. **Calling sum with two arguments.** Replace `/* function body here */` with:

```
subq    40,     %1,     %1
loadz   12,     %4
movq    %4,     32(%1)
loadz   30,     %4
movq    %4,     24(%1)
call    sum,    %3
movq    16(%1), %4
addq    40,     %1,     %1

movq    %4,     16(%2)
```

   The caller reserves 40 bytes: 16 bytes for the return address and old frame pointer, 8 bytes for the return value, and 16 bytes for the two arguments.
   Notice that the arguments are stored at offsets `32(%1)` and `24(%1)`, exactly where `sum` accesses them relative to `%2`. After the call, `main` retrieves the result from `16(%1)` and stores it in its own return-value slot at `16(%2)`.
5. **Assemble and run the program:**

```
ulmas -o ex1 ex1.s
ulm ex1
echo $?
```

   The exit code should be `42`.

Step 2 — Making Room for Local Variables

Let's artificially extend our example from Step 1 by introducing a local variable `r`:

```
fn sum(a: u64, b: u64): u64
{
    local r: u64;
    r = a + b;
    return r;
}

fn main(): u64
{
    return sum(12, 30);
}
```

The result is still `42`, but now sum needs space for a local variable on the stack.
We will not extend our compiler to support local variables just yet. Instead, we will implement this program directly in ULM assembly and complete our calling convention in preparation for the next worksheets.

Tasks

1. **Reserving space for a local variable.** Copy `ex1.s` to `ex2.s` and replace the implementation of `sum` with:

```
sum:
    movq    %3,     (%1)        # store return address
    movq    %2,     8(%1)       # store current frame pointer
    movq    %1,     %2          # set new frame pointer
    subq    8,      %1,     %1  # space for local variables

    // begin of function body
    movq    32(%2), %4
    movq    24(%2), %5
    addq    %4,     %5,     %4
    movq    %4,     -8(%2)      # store result in local variable

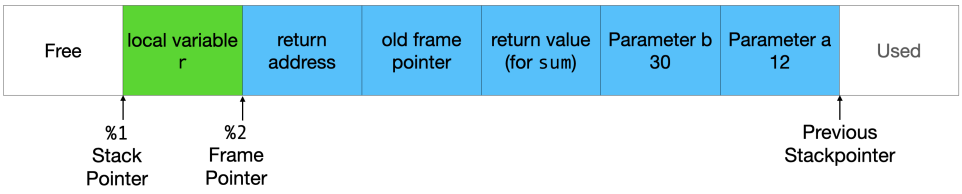
    movq    -8(%2), %4          # return local variable
    movq    %4,     16(%2)
    // end of function body

    movq    %2,     %1          # restore previous stack pointer
    movq    8(%1),  %2          # restore previous frame pointer
    movq    (%1),   %3          # restore return address
    ret     %3
```

After establishing the frame pointer, the additional instruction

```
subq    8,      %1,     %1  # space for local variables
```

reserves 8 bytes on the stack for our local variable `r`. The stack now looks as follows:



Notice how the function body implements our modified program:

```
movq    32(%2), %4
movq    24(%2), %5
addq    %4,     %5,     %4
movq    %4,     -8(%2)      # store result in local variable
```

First, the parameters a and b are loaded and added. The result is then stored in the stack slot reserved for r, at offset `-8(%2)`.

Next, we retrieve r and store its value in the return-value slot:

```
movq    -8(%2), %4          # return local variable
movq    %4,     16(%2)
```

Of course, this is rather wasteful: we load `r` back into the very register that already held its value! An optimizing compiler could eliminate these unnecessary memory accesses, giving us essentially the same assembly code as in Step 1.

But optimization is not our goal here. We want a simple, systematic way to generate correct code for local variables.

Reassemble and run `ex2.s`. The exit code should still be `42`.

2. **Defining the stack size later.** Copy `ex2.s` to `ex3.s`. We will now make two small changes to function `sum`:

(i) Replace

```
subq    8,      %1,     %1  # space for local variables
```

with

```
subq    .loc0,  %1,     %1  # space for local variables
```

(ii) and add the following directive immediately **after** the function:

```
.equ    .loc0,  8           # space for local variables
```

Notice that `.loc0` is not yet defined when the assembler encounters the `subq` instruction. This is perfectly fine! The assembler can handle references to symbols whose values are not yet known. If a symbol remains undefined, it may appear as an unresolved symbol in the object file. If its value is defined later, as in our example, the assembler can resolve the reference and generate the appropriate machine code.

The `.equ` directive defines `.loc0` to be `8`. It does not generate an instruction.

Reassemble and run `ex3.s`. The exit code should still be `42`.

Why is this useful for our compiler?

This will become our standard pattern. We will use a different symbol for each function (`.loc0`, `.loc1`, `.loc2`, ...), defining its value only after the function body.

Remember that our parser generates assembly code while parsing the function. When it emits the prologue, it does not yet know how many local variables the function will contain. With this trick, we can generate the prologue immediately and determine the required stack space only after the entire function definition has been parsed.

Step 3 — Bringing Back Our Symbol Table

Remember the symbol table from Worksheet 9? We haven't used it much lately, but now it's finally time to put it to work again! ;-)
Our code generator needs to assign stack offsets to function parameters and local variables. These offsets must be stored in the symbol table so that we can later access each variable relative to the frame pointer `%2`.
For now, we will focus on setting up this mechanism. We will generate the assembly code for sum, but leave its function body empty.

Create a test program. Create `xtest_gen_locals.abc` containing:

```
@ "gen.hdr"
@ "symtab.hdr"

fn main()
{
    // fn sum(a, b) {
    genFuncDefBegin("sum");
    genFuncDefParam("a");
    genFuncDefParam("b");

    // local r: u64;
    genAddLocal("r");

    // print offsets (relative to frame pointer)
    symtabPrintLocals();

    // }
    genFuncDefEnd();
}
```

Eventually, this program should generate the prologue and epilogue of sum and print the symbol table entries for `a`, `b`, and `r`, including their offsets relative to the frame pointer.

Extend the code generator interface. In `gen.hdr`, add the following declarations between `genFuncDefBegin` and `genFuncDefEnd`:

```
extern fn genFuncDefParam(ident: ->readonly char): bool;
extern fn genAddLocal(ident: ->readonly char): bool;
```

Both functions return `true` if the variable can be added successfully. If a variable with the same name already exists in the current scope, they return `false`. This will allow our compiler to report duplicate declarations later.

Add the implementations. For now, add the following stubs to `gen.abc`:

```
fn genFuncDefParam(ident: ->readonly char): bool
{
    return false;
}

fn genAddLocal(ident: ->readonly char): bool
{
    return false;
}
```

Build and test. Run make and verify that `xtest_gen_locals` builds successfully. Run the test program and inspect its output. For now, the generated assembly code should look like this:

```
.text
sum:
    subq    8,      %1,     %1
    movq    %3,     (%1)
    movq    %4,     8(%1)
    movq    (%1),   %3
    addq    8,      %1,     %1
    ret     %3
```

This is, of course, still our old prologue and epilogue from Worksheet 14. The symbol table is also empty, so `symtabPrintLocals()` has nothing to display.
But we now have everything in place to start working on `gen.abc`. In Step 4, we will implement the new calling convention from Steps 1 and 2, including frame pointers, parameter offsets, and space for local variables.

Step 4 — Updating the Code Generator (The Quick & Dirty Way)

We now have everything we need to implement our new calling convention in `gen.abc`. Let's start with a quick & dirty solution using a few internal global variables.

Tasks

1. **Keep track of function definitions and stack offsets.** In `gen.abc`, add the following global variables just before `genFuncDefBegin()`:

   ```
   global fnDefCount: int;
   global paramOffset: int = 24;
   global localVarOffset: int = -8;
   global inFnDef: bool;
   ```

   Here is what they are used for:
   - `fnDefCount` counts the number of completed function definitions, allowing us to generate the unique symbols `.loc0`, `.loc1`, etc.
   - `paramOffset` is the offset assigned to the next parameter, starting at `24`.
   - `localVarOffset` is the offset assigned to the next local variable, starting at `-8`.
   - `inFnDef` indicates whether we are currently generating a function definition, that means, `genFuncDefBegin()` has been called but `genFuncDefEnd()` has not. Nested function definitions are not allowed.
2. **Update the function prologue.** Replace `genFuncDefBegin()` with:

   ```
   fn genFuncDefBegin(name: ->readonly char)
   {
       assert(!inFnDef);
       inFnDef = true;
       setSegment(TEXT);
       printf("%s:\n", name);
       printf("\tmovq \t%%3, \t(%%1)\n");
       printf("\tmovq \t%%2, \t8(%%1)\n");
       printf("\tmovq \t%%1, \t%%2\n");
       printf("\tsubq \t.loc%d, \t%%1, \t%%1\n", fnDefCount);
       printf("\t// begin of function body\n");
   }
   ```

   As usual, much of the work is getting the escaping right: `\t` and `\n` are used for formatting, while `%%` is needed to print a literal `%`.
   Recall that the prologue already refers to `.loc0`, `.loc1`, etc., even though their values have not yet been defined. We will define them at the end of each function.
3. **Update the function epilogue.** Replace `genFuncDefEnd()` with:

   ```
   fn genFuncDefEnd()
   {
       assert(inFnDef);
       setSegment(TEXT);
       printf("\t// end of function body\n");
       printf("\tmovq \t%%2, \t%%1\n");
       printf("\tmovq \t8(%%1), \t%%2\n");
       printf("\tmovq \t(%%1), \t%%3\n");
       printf("\tret \t%%3\n");
       printf("\t.equ \t.loc%d, \t%d\n", fnDefCount++, -localVarOffset - 8);
       inFnDef = false;
       symtabFreeLocals();
       paramOffset = 24;
       localVarOffset = -8;
   }
   ```

   At the end of a function, we define the symbol `.locN` to specify how much stack space is needed for local variables. We then increment `fnDefCount`, clear the local symbol table, reset the offsets, and mark the function definition as complete. For now, `localVarOffset` will still be `-8`, so the generated `.equ` directive reserves 0 bytes.
4. **Build and inspect.** Rebuild `xtest_gen_locals` (did you remember all the required headers?) and inspect the generated assembly code. Does it match our new calling convention?

Step 5 — Managing Offsets in the Symbol Table

In this step, the code is provided. The goal is to see how the offsets that we manually managed in Steps 1 and 2 are now stored in the symbol table.

First, implement `genFuncDefParam`:

```
fn genFuncDefParam(ident: ->readonly char): bool
{
    assert(inFnDef);
    local added: bool = symtabAddLocal(ident, paramOffset);
    paramOffset += 8;
    return added;
}
```

Next, implement `genAddLocal`:

```
fn genAddLocal(ident: ->readonly char): bool
{
    assert(inFnDef);
    local added: bool = symtabAddLocal(ident, localVarOffset);
    localVarOffset -= 8;
    return added;
}
```

Build and inspect. Rebuild and run `xtest_gen_locals`. The generated output should now look like this:

```
       .text
sum:
       movq   %3,    (%1)
       movq   %2,    8(%1)
       movq   %1,    %2
       subq   .loc0,        %1,    %1
       // begin of function body
r with offset -8
b with offset 32
a with offset 24
       // end of function body
       movq   %2,    %1
       movq   8(%1),        %2
       movq   (%1),  %3
       ret    %3
       .equ   .loc0,        8
```

Verify that the offsets stored in the symbol table actually match those determined manually in Steps 1 and 2.

What's Next?

We now have the basic machinery for managing function parameters and local variables. Global variables will have to wait a little longer.

As our compiler grows, we will increasingly move back and forth between modules, extending existing functionality rather than implementing everything from scratch.

This is an important skill in software development: quickly finding your way into an unfamiliar part of a codebase, understanding what needs to change, making the change, and moving on.

Think of boxing: get in, do your job, and get out. And with practice, get faster at it.